



Manual Técnico Software e Infraestructura

	MACROPROCESO DE APOYO	CÓDIGO: ASIF020
	PROCESO GESTIÓN SISTEMAS Y TECNOLOGÍA	VERSIÓN: 9
	MANUAL TECNICO DE SOFTWARE E INFRAESTRUCTURA	VIGENCIA: 2024-12-05
		PAGINA: 2 de 30

TABLA DE CONTENIDOS

Tabla de contenido

Capítulo 1: Introducción	3
1.1 Propósito y Objetivos	3
1.2 Alcance del Sistema y del Documento	3
1.3 Audiencia	4
1.4 Definiciones, Acrónimos y Abreviaturas	4
Capítulo 2: Arquitectura General del Sistema	5
2.1 Visión General y Filosofía de Diseño	5
2.2 Arquitectura de componentes	5
Capítulo 3: Entorno Tecnológico	5
3.1 Aplicación Backend	5
3.2 Aplicación Frontend	6
3.3 Protocolos de Comunicación	6
Capítulo 4: Descripción de Módulos (Software Backend)	6
4.1 Capa de Control (API):	6
4.2 Capa de Lógica (Traduccion_models)	7
4.3 Capa de Abstracción de Hardware(cnc_driver)	8
Capítulo 5: Flujos de Datos y Procesos Clave	8
5.1 Flujo de Traducción y Vista Previa	8
5.2 Flujo de Impresión (G-Code y Comunicación Hardware)	9
Capítulo 6: Guía de Operaciones y Puesta en Marcha	10
6.1 Prerrequisitos de Software	10
Anexos	11

	MACROPROCESO DE APOYO	CÓDIGO: ASIF020
	PROCESO GESTIÓN SISTEMAS Y TECNOLOGÍA	VERSIÓN: 9
	MANUAL TECNICO DE SOFTWARE E INFRAESTRUCTURA	VIGENCIA: 2024-12-05
		PAGINA: 3 de 30

Capítulo 1: Introducción

1.1 Propósito y Objetivos

Este documento constituye la guía técnica para el software del proyecto "guía de integración de hardware para impresora braille como apoyo a personas con discapacidad visual". Su propósito fundamental es proporcionar una visión exhaustiva y detallada de la arquitectura del software, los flujos de datos y los protocolos de comunicación implementados para la conversión de documentos digitales en comandos físicos de impresión braille.

Los objetivos principales de este manual son:

- Describir la arquitectura de software (Frontend y Backend) y la pila tecnológica seleccionada (lenguajes, librerías).
- Documentar los flujos de datos clave, desde la carga de un archivo PDF por parte del usuario hasta la generación de G-Code y la traducción a Braille.
- Detallar el protocolo de comunicación y descubrimiento entre el software (Backend) y el hardware (ESP32).
- Servir como una referencia única para el personal técnico y desarrolladores encargados del mantenimiento, operación o extensión futura del sistema.

1.2 Alcance del Sistema y del Documento

El sistema es una aplicación de escritorio monousuario, diseñada para ejecutarse localmente en la computadora del operador. Su función principal es actuar como el centro de control de la impresora, gestionando la conversión de documentos (PDF) a un formato físico (impresión Braille).

El sistema integral se compone de dos componentes principales:

- Software (Aplicación de Escritorio): Proporciona la interfaz gráfica de usuario (Frontend) para la carga de archivos y la visualización de la traducción. Incluye un servidor local (Backend) que ejecuta todo el procesamiento, la lógica de traducción y la comunicación con el hardware.
- Hardware (Impresora Braille): Un dispositivo de hardware basado en el microcontrolador ESP32. Es el responsable de recibir comandos G-Code a través de la red local (WiFi) y traducirlos en movimientos mecánicos y acciones de punzonado.
- Este manual cubre exclusivamente el diseño, la arquitectura y la operación del componente de Software. El diseño y la lógica del firmware de la ESP32 se

	MACROPROCESO DE APOYO	CÓDIGO: ASIF020
	PROCESO GESTIÓN SISTEMAS Y TECNOLOGÍA	VERSIÓN: 9
	MANUAL TECNICO DE SOFTWARE E INFRAESTRUCTURA	VIGENCIA: 2024-12-05
		PAGINA: 4 de 30

documentarán en un manual de hardware separado.

1.3 Audiencia

Este documento está dirigido a perfiles técnicos que requieran una comprensión profunda del funcionamiento interno del software:

- **Desarrolladores de Software:** Utilizarán este manual para comprender la arquitectura del código, la responsabilidad de cada módulo, la lógica de traducción y cómo el sistema se integra con el hardware.
- **Ingenieros o Técnicos de Integración:** Consultarán este documento para entender el protocolo de comunicación (mDNS, Telnet/Socket), la secuencia de comandos G-Code y los requisitos de red para la correcta operación del sistema.

1.4 Definiciones, Acrónimos y Abreviaturas

La siguiente tabla define los términos y acrónimos utilizados a lo largo del documento.

Término	Descripción
API	Application Programming Interface; interfaz que permite la comunicación entre el Frontend y el Backend.
BACKEND	Servidor local (basado en Flask) que ejecuta la lógica de negocio (traducción, generación de G-Code).
ESP32	Microcontrolador con conectividad WiFi utilizado como unidad de control principal de la impresora Braille.
FRONTEND	Aplicación de escritorio (basada en Electron/JavaScript) con la que interactúa el usuario.
G-CODE	Lenguaje de control numérico. Es el estándar de comandos utilizado para enviar instrucciones de movimiento (G0, G1) y acciones (M3, M5) a la impresora.
GRBL	Firmware de código abierto para control de movimiento que se ejecuta en la ESP32 y gestiona la interpretación de G-Code.
mDNS (Zeroconf)	Protocolo de descubrimiento de servicios en red local. Permite al backend descubrir la dirección IP de la impresora automáticamente usando un nombre, sin necesidad de configuración manual
PyMuPDF (fitz)	Librería de Python utilizada para la extracción de contenido (texto) de archivos PDF.
Socket (TCP/IP)	Protocolo de comunicación utilizado para establecer una conexión directa y persistente (tipo Telnet) con el puerto 23 de la impresora, a través del cual se transmiten los comandos G-Code.

	MACROPROCESO DE APOYO	CÓDIGO: ASIF020
	PROCESO GESTIÓN SISTEMAS Y TECNOLOGÍA	VERSIÓN: 9
	MANUAL TECNICO DE SOFTWARE E INFRAESTRUCTURA	VIGENCIA: 2024-12-05
		PAGINA: 5 de 30

Capítulo 2: Arquitectura General del Sistema

2.1 Visión General y Filosofía de Diseño

La arquitectura del sistema se basa en un modelo Cliente-Servidor Local. Este diseño proporciona una clara separación de responsabilidades entre la interfaz de usuario y la lógica de procesamiento, todo contenido dentro de la máquina del usuario.

- **Cliente (Frontend):** Es una aplicación de escritorio basada en tecnología web que gestiona la interacción con el usuario.
- **Servidor (Backend):** Es una API REST ligera basada en Python/Flask que se ejecuta localmente.

La filosofía de diseño es el procesamiento al vuelo (on-the-fly), lo que significa que procesa la información en el mismo instante en que la necesita. El sistema es sin estado (stateless), por lo que no requiere una base de datos. Los archivos PDF no se almacenan; se cargan en memoria, se procesan, se traducen a G-Code, y dichos comandos se transmiten directamente al hardware.

2.2 Arquitectura de componentes

La interacción entre los componentes del sistema sigue un flujo lógico y desacoplado:

- **Frontend (Cliente):** Responsable de renderizar la interfaz gráfica, gestionar la selección de archivos PDF y realizar solicitudes HTTP a la API del backend.
- **Backend (Servidor API):** Expone endpoints HTTP para procesar las solicitudes. Contiene toda la lógica de negocio y actúa como orquestador, invocando los módulos de traducción y comunicación.
- **Hardware (Impresora):** El dispositivo físico que recibe los comandos. Se comunica exclusivamente con el Backend a través de protocolos de red WiFi (mDNS para descubrimiento, Socket TCP para comandos).

Capítulo 3: Entorno Tecnológico

Este capítulo desglosa las tecnologías específicas seleccionadas para construir y operar el sistema:

3.1 Aplicación Backend

- **Lenguaje de Programación:** Python 3.
- **Framework de API:** Flask (Micro-framework web para la creación de la API REST).
- **Servidor WSGI:** Werkzeug (Servidor de desarrollo local de Flask).

	MACROPROCESO DE APOYO	CÓDIGO: ASIF020
	PROCESO GESTIÓN SISTEMAS Y TECNOLOGÍA	VERSIÓN: 9
	MANUAL TECNICO DE SOFTWARE E INFRAESTRUCTURA	VIGENCIA: 2024-12-05
		PAGINA: 6 de 30

- **Librerías Clave:**

- **flask_cors:** Habilita la comunicación entre el Frontend y el Backend (CORS).
- **fitz (PyMuPDF):** Utilizada para la extracción robusta de texto de archivos PDF.
- **zeroconf:** Implementa el descubrimiento de servicios mDNS para localizar la impresora en la red.
- **socket:** Utilizada para la comunicación TCP/IP de bajo nivel con el firmware GRBL.

3.2 Aplicación Frontend

- **Tecnologías Base:** HTML5, CSS3, JavaScript (ES6+).
- **Entorno de Ejecución:** Aplicación de escritorio con Electron.
- **Librerías Clave (JavaScript):**
 - **Fetch API (Nativa de JS):** Se utiliza para realizar todas las solicitudes asíncronas (POST) a los endpoints /traducir y /imprimir del backend.

3.3 Protocolos de Comunicación

- **Frontend a Backend:** Se utiliza HTTP sobre localhost. El frontend envía solicitudes POST al backend:
 - **multipart/form-data:** Para enviar el archivo PDF en la ruta /traducir.
 - **application/json:** Para enviar el texto a imprimir en la ruta /imprimir.
- **Backend a Hardware (Conexión):** Se utiliza mDNS (Zeroconf). El backend emite una consulta mDNS para el hostname "impresorabril" para resolver su dirección IP.
- **Backend a Hardware (Comandos):** Se utiliza una conexión TCP/IP (Socket) directa al puerto 23 (Telnet) de la impresora. El backend envía comandos G-Code como cadenas de texto ASCII y espera una respuesta "ok" de GRBL antes de enviar el siguiente comando.

Capítulo 4: Descripción de Módulos (Software Backend)

El código fuente del backend, está estructurado siguiendo un patrón de separación de responsabilidades (Controladores y Modelos).

4.1 Capa de Control (API):

Esta capa actúa como el punto de entrada de la API y orquesta la lógica de la aplicación. Define los endpoints HTTP (/traducir y /imprimir) que el Frontend consume. No contiene

	MACROPROCESO DE APOYO	CÓDIGO: ASIF020
	PROCESO GESTIÓN SISTEMAS Y TECNOLOGÍA	VERSIÓN: 9
	MANUAL TECNICO DE SOFTWARE E INFRAESTRUCTURA	VIGENCIA: 2024-12-05
		PAGINA: 7 de 30

lógica de negocio, sino que delega las tareas a los módulos correspondientes en la capa de Modelos.

- **Endpoint POST /traducir:**

1. Recibe un archivo PDF (FormData).
2. Utiliza fitz (PyMuPDF) para extraer el texto plano del documento.
3. Invoca a `traduccion_models.texto_a_unicode()` para obtener una representación visual del Braille.
4. Devuelve un JSON al frontend con el texto original y la cadena Unicode para la vista previa.

- **Endpoint POST /imprimir:**

1. Recibe un JSON con el texto a imprimir (obtenido del flujo de traducción).
2. Invoca a `traduccion_models.texto_a_braille()` para obtener las matrices de puntos.
3. Invoca a `traduccion_models.montar_pagina_y_generar_gcode()` para crear el archivo `impresion.gcode` localmente.
4. Invoca a `cnc_driver.descubrir_impresora_mdns()` para encontrar la IP del hardware.
5. Invoca a `cnc_driver.enviar_gcode_a_impresora()` para transmitir el archivo G-Code.
6. Devuelve un JSON de éxito al frontend.

4.2 Capa de Lógica (Traduccion_models)

Contiene toda la lógica de conversión de texto a Braille y de Braille a G-Code.

- **CHAR_TO_MATRIX:** Diccionario Python que sirve como la base de datos de traducción. Mapea caracteres (incluyendo mayúsculas y números) a sus matrices Braille 3x2.
- **texto_a_braille(texto):** Función que implementa la lógica de traducción. Itera sobre el texto de entrada y maneja las reglas del Braille, como la inserción de prefijos CAP (mayúscula) y NUM (número) antes de los caracteres correspondientes.
- **montar_pagina_y_generar_gcode:** Esta función realiza la estructuración de la página:
 - Genera el archivo `impresion.gcode` que contiene los comandos de movimiento (G0 X... Y...). Esta función no genera G-Code de punzonado (M3/M5). En su lugar, inserta un marcador de texto especial (";PUNCH") en el archivo G-Code. Esto desacopla la lógica de traducción de la lógica de hardware.

	MACROPROCESO DE APOYO	CÓDIGO: ASIF020
	PROCESO GESTIÓN SISTEMAS Y TECNOLOGÍA	VERSIÓN: 9
	MANUAL TECNICO DE SOFTWARE E INFRAESTRUCTURA	VIGENCIA: 2024-12-05
		PAGINA: 8 de 30

- Implementa la lógica de salto de palabra (word wrap) y salto de línea.

4.3 Capa de Abstracción de Hardware(cnc_driver)

Este módulo encapsula toda la comunicación directa con la impresora.

- **descubrir_impresora_mdns(hostname):** Utiliza la librería zeroconf para enviar una consulta mDNS a la red local. Espera una respuesta del hostname y devuelve su dirección IP.
- **enviar_gcode_a_impresora(host, archivo_gcode):**
 1. Establece una conexión socket TCP al host (IP) en el puerto 23.
 2. Espera el mensaje de bienvenida de GRBL.
 3. Envía comandos de inicialización (\$X para desbloquear, M5 S1 para asegurar que el punzón esté arriba).
 4. Lee el archivo_gcode línea por línea.
 5. Si la línea es un comando G-Code; la envía por el socket, seguida de \n, y espera la respuesta "ok" de GRBL antes de continuar.
 6. Si la línea es el marcador ";PUNCH"; el driver ejecuta una secuencia de comandos de punzonado, envía M3 S1 (bajar punzón), espera "ok", envía M5 S1 (subir punzón), y espera "ok".

Capítulo 5: Flujos de Datos y Procesos Clave

5.1 Flujo de Traducción y Vista Previa

Este proceso se activa cuando el usuario carga un archivo PDF y presiona el botón "Traducir". El objetivo es extraer el texto y mostrar una vista previa en Braille Unicode en la interfaz.

1. El Usuario selecciona un archivo PDF en el Frontend.
2. El Frontend (renderer.js) crea un objeto FormData, adjunta el archivo y envía una solicitud POST /traducir al Backend.
3. El Backend (api.py) recibe la solicitud. El controlador extrae el PDF y utiliza la librería fitz para leer y extraer todo el texto plano (texto_total).
4. El controlador pasa texto_total al módulo traduccion_models (función texto_a_unicode).
5. El módulo traduccion_models procesa el texto y lo convierte en una cadena de caracteres Braille visibles.
6. El Backend responde al Frontend con un JSON que contiene la cadena Unicode y el texto original extraído.
7. El Frontend recibe el JSON, almacena el texto original en una variable para el

	MACROPROCESO DE APOYO	CÓDIGO: ASIF020
	PROCESO GESTIÓN SISTEMAS Y TECNOLOGÍA	VERSIÓN: 9
	MANUAL TECNICO DE SOFTWARE E INFRAESTRUCTURA	VIGENCIA: 2024-12-05
		PAGINA: 9 de 30

paso de impresión y muestra la cadena Unicode en el área de vista previa.

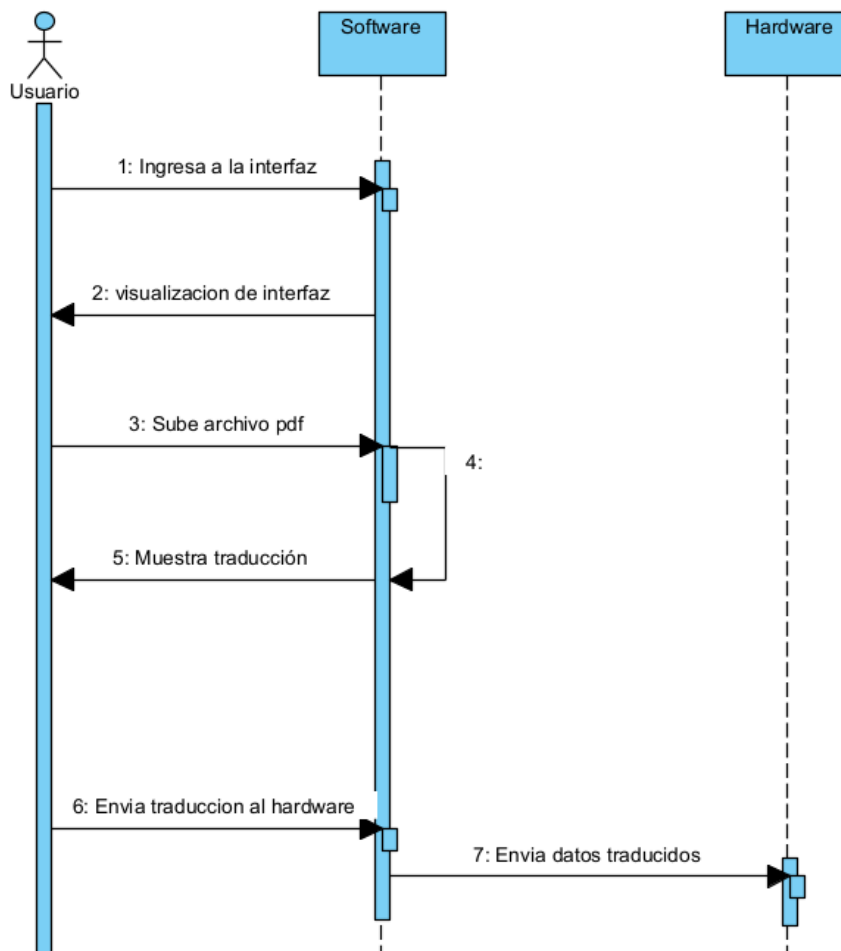


Imagen 1. Diagrama de Secuencia

5.2 Flujo de Impresión (G-Code y Comunicación Hardware)

Este proceso se activa cuando el usuario presiona "Imprimir":

1. El Usuario presiona "Imprimir" en el Frontend.
2. El Frontend (renderer.js) toma el texto original guardado del flujo anterior, lo empaqueta en un JSON y envía una solicitud POST /imprimir al Backend.
3. El Backend (api.py) recibe el JSON.
4. **Fase 1: Generación de G-Code.**
 - El controlador envía el texto a `traduccion_models.texto_a_braille()` para obtener las matrices lógicas.

	MACROPROCESO DE APOYO	CÓDIGO: ASIF020
	PROCESO GESTIÓN SISTEMAS Y TECNOLOGÍA	VERSIÓN: 9
	MANUAL TECNICO DE SOFTWARE E INFRAESTRUCTURA	VIGENCIA: 2024-12-05
		PAGINA: 10 de 30

- Luego, pasa las matrices a `traduccion_models.montar_pagina_y_generar_gcode()`.
- El modelo escribe el archivo `impresion.gcode` en el disco del servidor, conteniendo comandos G0... y marcadores ";PUNCH".

5. Fase 2: Descubrimiento de Hardware.

- El controlador llama a `cnc_driver.descubrir_impresora_mdns("impresorabrilie")`.
- El `cnc_driver` emite una consulta mDNS a la red.
- El Hardware (ESP32) responde a la consulta con su dirección IP.

6. Fase 3: Envío de Comandos.

- El controlador llama a `cnc_driver.enviar_gcode_a_impresora()` con la IP y el nombre del archivo.
- El `cnc_driver` abre un socket TCP al Hardware en el puerto 23.
- El `cnc_driver` envía comandos de inicialización y espera "ok" por cada uno.
- Inicia un bucle de lectura del archivo `impresion.gcode`:
 - Si lee "G0 X10.00 Y2.00", lo envía al Hardware. El Hardware ejecuta el movimiento y responde "ok".
 - Si lee ";PUNCH", el `cnc_driver` ejecuta su secuencia interna: envía M3 S1, espera "ok", envía M5 S1, espera "ok".
 - El bucle continúa hasta que el archivo se termina.

7. Fase 4: Finalización.

- El `cnc_driver` cierra el socket y retorna éxito al controlador.
- El Backend (`api.py`) responde al Frontend con un JSON de éxito.

Capítulo 6: Guía de Operaciones y Puesta en Marcha

6.1 Prerrequisitos de Software

- **Python:** Se requiere Python 3.8 o superior.
- **Dependencias de Python:** Es necesario instalar las librerías de Python listadas en `requirements.txt` o manualmente.
- **Frontend:** Una aplicación Electron compilada o un navegador web moderno (para `index.html`) capaz de ejecutar `renderer.js`

	MACROPROCESO DE APOYO	CÓDIGO: ASIF020
	PROCESO GESTIÓN SISTEMAS Y TECNOLOGÍA	VERSIÓN: 9
	MANUAL TECNICO DE SOFTWARE E INFRAESTRUCTURA	VIGENCIA: 2024-12-05
		PAGINA: 11 de 30

Anexos

DATOS DE CONTACTO SOPORTE APLICACIÓN

Angie Paola Cárdenas Montaña
Correo: apaolacardenas@ucundinamarca.edu.co

Yesica Lorena Gonzalez Riveros
Correo: ylorenagonzalez@ucundinamarca.edu.co



	MACROPROCESO DE APOYO		CÓDIGO: ASIF020
	PROCESO GESTIÓN SISTEMAS Y TECNOLOGÍA		VERSIÓN: 9
	MANUAL TECNICO DE SOFTWARE E INFRAESTRUCTURA		VIGENCIA: 2024-12-05
			PAGINA: 8 de

CONTROL DE CAMBIOS				
VERSIÓN	FECHA DE APROBACIÓN			DESCRIPCIÓN DEL CAMBIO
	AAAA	MM	DD	
1	2025	10	10	Emisión del documento.

ELABORÓ				
NOMBRES Y APELLIDOS		CARGO		
Angie Paola Cárdenas Montaña		Estudiante Pregrado		
Yesica Lorena Gonzalez Riveros		Estudiante Pregrado		
REVISÓ				
NOMBRES Y APELLIDOS		CARGO		
Ing. Juan Pablo Pedroche		Docente		
APROBÓ (GESTOR RESPONSABLE DEL PROCESO)				
NOMBRES Y APELLIDOS	CARGO	FECHA		
		AAAA	MM	DD
Ing. Juan Pablo Pedroche	Docente	2025	10	27